

Scintilla — Known Issues & Setup Notes

Last checked: 2026-09-02. Everything below was verified by actually running the app.

Setup status

Thing	Status
Azure credentials in <code>scintilla-backend/.env</code>	Done — token acquisition verified
Graph permissions (<code>Files.ReadWrite.All</code> , <code>Sites.ReadWrite.All</code> , <code>User.Read.All</code>)	Already granted
<code>MS_DRIVE_USER_ID</code>	Still empty — must be filled in
Excel workbook + named table	Not verified — see Issue 6

The one value still missing

Open `scintilla-backend/.env` and set:

```
MS_DRIVE_USER_ID=someone@snuchennai.edu.in
```

This is the email of the account whose OneDrive holds the submissions. Until it is set, every upload fails.

To check it worked, run from `scintilla-backend/`:

```
node -e "require('dotenv').config();
require('./src/services/oneDriveService').checkHealth().then(ok =>
console.log(ok ? 'DRIVE OK' : 'DRIVE FAILED'))"
```

Issue 1 — The backend has no authentication (most important)

What's wrong: The Microsoft sign-in only happens in the browser. The result is stored in `sessionStorage` and never sent to the server. The server trusts whatever name and email it receives in the POST body.

Anyone can skip the login page entirely and write files into your OneDrive:

```
curl -X POST http://localhost:3000/api/v1/submit \
-F "name=Not A Student" -F "email=fake@snuchennai.edu.in" \
-F "course=B.Com" -F "year=2" \
```

```
-F 'submissions=[{"category":"poetry","title":"x","description":""}]' \
-F "files=@anything.pdf"
```

I ran this. It goes straight through validation.

Where to change:

1. `submit.html` line 735 — send the Microsoft token with the request:

```
const resp = await fetch(`${API_BASE}/api/v1/submit`, {
  method: 'POST',
  headers: { 'Authorization': `Bearer ${accessToken}` },
  body: formData
});
```

The token comes from MSAL. Currently `scintilla_login.html` throws it away — around line 430 it saves only name/email/expiry into `sessionStorage`. Save `response.accessToken` too.

2. Add a new file `scintilla-backend/src/middleware/auth.js` that validates that token against the tenant's public keys (`jose` or `jwt-rsa` package), confirms the tenant ID matches `5beb351c-...`, and confirms the email ends in `@snuchennai.edu.in`.
3. `scintilla-backend/src/routes/submit.js` line 60 — add it to the chain:

```
router.post('/', submitLimiter, requireAuth, uploadMiddleware,
  validateSubmission, submissionController.handleSubmit);
```

4. Then take `name` and `email` from the verified token, not from `req.body` (`submissionController.js` line 32).

Issue 2 — Wrong Node version declared; app crashes on Node 18 or 20

What's wrong: `package.json` says the app runs on Node 18+. It does not. The `file-type` package (v21) is ESM-only and needs Node 20+, and the `require()` of it in `submissionController.js` only works from Node 22.12 onward. On Node 18 or 20 the server crashes at startup.

Local machine is on Node 25, so it works here. A deploy host defaulting to Node 18 or 20 will not.

Where to change: `scintilla-backend/package.json` line 14:

```
"node": ">=22.12.0"
```

Then make sure the host (Render / Railway / Azure / whatever) is set to Node 22 or newer.

Issue 3 — If OneDrive is down at startup, uploads stay broken until restart

What's wrong: `server.js` line 13 calls `ensureFolders()` once at boot to populate an in-memory map of category folder IDs. If that call fails, the server still starts, but the map stays empty — so every upload afterwards fails with "Category folder not found", forever, until someone manually restarts the process.

This is exactly what happened during testing.

Where to change: `scintilla-backend/src/services/oneDriveService.js` around line 148. Instead of throwing when the folder is missing, look it up on demand:

```
let folderId = categoryFolders.get(catKey);
if (!folderId) {
  folderId = await ensureFolder(category.charAt(0).toUpperCase() +
category.slice(1));
  categoryFolders.set(catKey, folderId);
}
```

Issue 4 — Excel column order in the comment does not match the code

What's wrong: The comment at `excelService.js` line 8 says the table columns are:

```
Name, Email, Course, Year, Category, Title, Description, FileLink,
Timestamp    (9 columns)
```

But the code at `submissionController.js` line 91 actually sends:

```
submissionId, timestamp, name, email, course, year, category, title,
description, fileLink    (10 columns)
```

If the workbook is built from the comment, every row lands in the wrong column and the append fails on the column count.

Where to change: `scintilla-backend/src/services/excelService.js` line 8 — fix the comment to match the code. The code order is the correct one.

Issue 5 — CORS rejections return 500 instead of 403

What's wrong: `security.js` line 45 throws a plain `Error`, which has no status, so the global error handler defaults to 500. A blocked origin should be a client error, not a server error.

Where to change: `scintilla-backend/src/middleware/security.js` line 45:

```
const err = new Error("Not allowed by CORS");
err.status = 403;
cb(err);
```

Issue 6 — The Excel workbook must be created by hand first

What's wrong: Nothing in the code creates the workbook. If it does not already exist, every submission uploads the file to OneDrive successfully and then fails on the Excel step, returning 207 Multi-Status.

What to do: In the OneDrive of whichever account you put in `MS_DRIVE_USER_ID`, create:

```
ScintillaSubmissions/Submissions.xlsx
```

In it, make a header row with these 10 columns in this exact order:

```
SubmissionID | Timestamp | Name | Email | Course | Year | Category | Title
| Description | FileLink
```

Select the header row plus one empty row, Insert > Table, tick "My table has headers", then rename the table to `Submissions` (Table Design > Table Name). The name must match `MS_EXCEL_TABLE_NAME` in `.env`.

Before going live

In `scintilla-backend/.env`:

- `STATUS_BEARER_TOKEN` — currently the placeholder `CHANGE_ME_local_dev_token`. Replace with a real random secret.
- `ALLOWED_ORIGINS` — currently localhost only. Add the real site domain.
- `NODE_ENV=production`.

In `submit.html` line 732 — the non-localhost branch assumes the API is on the same hostname as the site with no path prefix. If the backend lives somewhere else, hardcode its URL there.

Rotate `MS_CLIENT_SECRET` in the Azure portal before launch — the current one has been pasted into chat logs and should not be considered private.

Running locally

```
# terminal 1 — backend
cd scintilla-backend
npm install
npm start                                # port 3000
```

```
# terminal 2 – frontend  
python3 -m http.server 5500 # from the repo root
```

Open <http://localhost:5500/index.html>

Note: the frontend must be served over HTTP, not opened as a `file://` path — Microsoft sign-in needs a real origin. For sign-in to work locally, http://localhost:5500/scintilla_login.html must be registered as a redirect URI on the Azure app registration.

To skip the login page while testing, run this in the browser console and then go to [submit.html](#):

```
sessionStorage.setItem('scintilla_auth', JSON.stringify({  
  name: 'Test', email: 'test@snuchennai.edu.in', expiresAt: Date.now() +  
  7200000 })))
```